Multimedia Tools and Applications (2020) 79:26245–26263
https://doi.org/10.1007/s11042-020-09239-0

Pseudoinverse learning autoencoder with DCGAN for plant diseases classification

Mohammed A. B. Mahmoud[1] · **Ping Guo**[2] · **Ke Wang**[3]

Received: 3 September 2019 / Revised: 10 June 2020 / Accepted: 24 June 2020 /
Published online: 13 July 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract
Pest infestation of crops and plants impacts agricultural development. Generally, farmers or specialist observe the plants with the naked eye to recognise and diagnose ailments. However, this technique can be time-consuming, costly and inexact. In contrast, auto-detection using image processing methods gives fast and precise results. This paper introduces a new plant disease identification model predicated on leaf image classification that employs a deep convolutional generative adversarial network (DCGAN) along with a classifier identified by multilayer perceptron (MLP) neural networks trained with a pseudoinverse learning autoencoder (PILAE) algorithm. The DCGAN performes two tasks: (1) synthesis of the minor class images to overcome the issue of imbalance in the dataset and (2) extracting deep features of all images within the dataset. The PILAE training procedure is not required to identify the learning control variables or indicate the number of hidden layers. Consequently, the PILAE classifier can fulfil exceptional execution with regard to training efficiency and reliability. Empirical results from PlantVillage dataset possess demonstrated how the presented method yields positive results with other models and reasonably minimal complexly.

Keywords Plant diseases · DCGAN · Pseudoinverse learning

1 Introduction

Plant diseases can cause substantial damage to agriculture plants by significantly reducing yield [27]. Pests are a typical exemplary of disease that may severely impact early plant

This work is fully supported by the grants from the National Natural Science Foundation of China (NSFC) (61375045), and the Joint Research Fund in Astronomy (U1531242) under cooperative agreement between the NSFC and Chinese Academy of Sciences (CAS).

✉ Mohammed A. B. Mahmoud
mahmoud.mohammed@bit.edu.cn

1 School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China

2 School of Systems Science, Beijing Normal University, Beijing, China

3 Department of Electrical and Computer Engineering, University of Michigan-Dearborn, Dearborn, MI, USA

development [9]. Similarly, in humid weather, pests can cause very harmful disease in more mature plants and can impact plant leaves, fruit and stems [9]. Protecting plants from ailments is key to guarantee the quality and quantity of crops [11]. An effective protection approach should focus on early recognition of the condition to enable the choice of ideal treatment at the proper time to prevent it from spreading [2]. Generally, this detection can be achieved by professionals with academics knowledge strengthened by working experience on symptoms of and factors behind diseases [9]. Moreover, those professionals must monitor crops consistently in order to prevent the growth of disease.

This continuous monitoring is a difficult and time-consuming process for humans, making automation in the plant diseases recognition and identification necessary to protect crops [11]. Several analyses [1, 2, 15, 41] have already been proposed to identify and classify plant diseases utilizing machine learning and image processing. These strategies attempt to develop disease classifiers by employing images extracted from the plants. These classifiers are derived from hand-crafted features created by specialists to extract relevant details for classification of the image. Because of this, these classifiers have the disadvantages of being less automated due to their dependency on hand-crafted features [35]. Furthermore, the classifier are trained utilizing images labeled by specialists, and accumulating these labelled images is very costly because the labelling is performed manually. These problems with data selection have forced the prior studies to utilize small datasets to train and examine classifiers [1, 2, 15, 41]. The use of smaller labelled datasets is a limiting factor in deep learning (DL) strategies and can also result in overfitting. In [28], 54,306 images of healthy and diseased plant leaves were collected, making the training of DL classifier attainable. This new development produced more precise classifiers compared to traditional machine learning methods [11, 12, 16, 21, 30, 37, 40–42, 52, 69].

The core idea of DL is using first stemming forthright representations to learn hierarchical feature representation and then proceeding to unremittingly develop more complex ones from the preceding level. Compared with the handcrafted models, DL design is capable of encoding data that is multilevel from their initial simple nature to a more intricate one. Therefore, for feature learning, this method is highly encouraging because (1) it does not necessitate ground truth; (2) it induces intricate non-linear connections using deep architecture that is hierarchical in nature; (3) it does not depend on chosen handcrafted features but rather is completely driven by data [32] and (4) owing to its trained hierarchical deep network, it is capable of effectively and rapidly determining the feature representation of low-level images.

Normal oversampling techniques are inspired by the synthetic minority over-sampling technique (SMOTE) algorithm [13], which creates artificial samples over the line segment that connects minority class samples. Specifically, SMOTE creates the same number of synthetic samples for each original minority sample and does so without regard to neighboring samples, which increases the incidence of overlapping between classes [65]. A direct method to create data would be generative approach [23, 45] that replicates the current data distribution.

Zhang Jing [72] utilized DCGAN for feature extraction in the retrieval task for Hyperspectral Images. In [73], the authors used DCGAN to extract features as an alternative to hand-crafted features. After training the network, they utilized the discriminator's features from all layers as signature features and then train a hybrid user-independent/ user-dependent classifier. [38] utilized the DCGAN as features extracted for traffic sign images followed by the pseudoinverse learning autoencoder (PILAE) classifier. DCGAN is also

used for synthetic oversampling methods [8, 29, 39, 44, 54, 72]. Thus, The DCGAN can be used as feature extraction and generated samples for the proposed method.

A pseudoinverse learning algorithm PIL) [24, 63], it is a multilayer perceptron (MLP) learning algorithm composed of stacked generalization connected such that it dominates the neural networks (NN) degradation predictive accuracy. Its structure possesses an identical number of hidden neurons as the number of samples that are to be learned. PIL overcomes learning errors by performing the addition of hidden layers. It had been fully automated, feed-forward and does not contain critical user subordinate parameters, for example, learning rate, the maximum epoch and momentum constants. PIL has been demonstrated to be an efficient algorithm and by far much better than the standard back propagation (BP) and other algorithms of gradient descent. Wang et al. [64] asserted that PILAE was a fully and fast automated framework that uses deep neural networks to train stacked autoencoders. PILAE trains the stacked autoencoder by embracing the PIL algorithm with a low-rank approximation; PILAE is not a gradient descent method and does not have the shortcoming of gradient vanishing. It also does not have the problem of saturation activation as a matrix that is multiplied by its pseudoinverse.

In summary, the contributions of this paper are threefold. First, extract deep features using DCGAN, which can learn powerful feature representation in an unsupervised manner; sampling the minor classes to overcome the imbalanced problem which has a great effect in prediction accuracy. Second, randomly initialize weight variables in the BP algorithm and create the delta learning rule to update the weight matrix. However, in PILAE algorithm, weight parameters are calculated with the pseudoinverse solution and do not have to be modified further, thus having an important effect on the implementation of the model. Third, investigate the performance of the introduced model on a standard dataset from PlantVillage [28] leading to a comparison of the recognition rate of peer opponents and analysis the evolution performance of the proposed model.

The paper proceeds to evaluate Plant diseases classification related issued in section 2. Section 3 provides a detailed description of the methods used in the proposed model. Section 4 presents the proposed model, implementation details, experimental results and the discussion. Section 5 concludes the paper.

2 Related works

Several methods are employed within the agricultural field like the investigation of plant pests and disease. DL has also been applied in addition to image processing approaches. Traditional machine learning methods have already been extensively adopted inside the agricultural field.

In [40], the authors applied a DL approach to develop a mobile-facilitated disease detection technique. The authors used CNN to train their design, employing datasets of 54,306 images of infected and healthy leaves. CNN had been trained to recognize 26 diseases and 14 plant species. The authors applied two structures: GoogLeNet [56] and AlexNet [33]. The model has the capacity to achieve a precision of 99.35%. Although this method produced state-of-the-art results, it implemented inadequately when it was tested on images obtained under different situation.

[52] implemented Deep CNN for the development of plant disease recognition framework using leaf images. The framework could distinguish 14 kinds of plant disease from

healthy leaves. Furthermore, it was able to identify plants using their surroundings. The suggested method typically obtained a 96.3% prediction accuracy in their experimental analysis. In work presented by Dyrmann et al. [17], the authors suggested a method can identify plant and weeds species utilizing RGB images. The authors utilized CNN tested over 10,413 images with 22 kinds of crops and weeds. CNN was able to attain a classification precision of 86.2%. This model had difficulty in classifying some plant species, which is thought to have been because of the low number of training images for those species. Sa et al. [48] proposed DeepFruits model, which is utilized for recognition of fruits. The authors implemented the more rapid R-CNN [46] method, known as the multi-modal Faster R-CNN. The authors trained the presented model and achieved a noticeable improvement from the prior work, achieving 83.8% accuracy. The authors retrained the model to execute the detection of seven fruits, with the entire process of annotating and training the new model requiring 4 houres per fruit. In another study, Athanikar and Badar [5] utilized neural network (NN) to classify the potato leaf image as either diseased or healthy. The results demonstrated that back propagation neural network (BPNN) could efficiently identify the disease areas and classify this disease type, with an accuracy of 92%.

In another study, Wang et al. [66] performed an analysis to identify a strategy to realise image identification of plant disease. Four forms of NN have been utilized to differentiate between wheat leaf rust and wheat stripe rust and between grape downy mildew and grape powdery mildew, predicated on features extracted from the disease images. The results demonstrated that recognition and examination of the plant disease could potentially be effectively obtained employing NN predicated on image processing. Samanta et al. [49] proposed strategy to identify potato scab disease. The dataset used in the model was accumulated from diverse potato fields and was prepared for improvement using image segmentation to concentrate on regions (condition areas). Finally, an evaluation of the mark regions (condition areas) predicated on a histogram approach was used to determine the phase of the condition. [11] employed CNN as a learning method which utilized images immediately. The authors used DL as a framework for classifying plant diseases, concentrating on leaf images. This model is used to recognize and localize regions within the leaf. The method achieved a 99.18% accuracy.

Nazki et al. [43] designed activation reconstruction GAN (AR-GAN), a model that employed GANs in an image translation framework to synthetically augment plant disease samples and further improve the efficiency of the plant disease recognition task employing deep CNN. The main limitation of this model is the long training time (up to three days). Zhu et al. [75] utilized supervised conditioning to train a GAN to make synthetic images of Arabidopsis plants. In both of these studies, the synthetic GAN images were utilized for data augmentation and, thus, to raise the fulfillment on leaf counting. Giuffrida et al. utilized GAN-generated samples to train a NN for Arabidopsis leaf counting [60]. Similarly, Arsenovic et al. employed StyleGAN to generate training samples for plant disease sample classification [4]. Tian et al., in [57], demonstrated the use of cycle GAN for preferable augmentation of diseased apple samples to boost the size of the dataset. According to [67], a hyperspectral proximal-sensing model based on the newest deep learning method called outlier removal ouxiliary classifier generative adversarial nets (OR-AC-GAN) was presented in order to detect the tomato plant disease before clear symptoms appeared.

The methods mentioned above suffer from data scarcity, high computational complexity or both. In contrast, the proposed DCGAN-PILAE method overcomes these limitations by using DCGAN to generate more data samples and applying the PILAE classifier to provide exceptional execution with regard to training efficiency and reliability.

3 Methods

3.1 Deep convolution generative adversarial network (DCGAN)

DCGAN [45] is an image generation model that is composed of two neural networks, a discriminative network D and a generative network G. The G network is a transposed convolutional neural network that does not possess fully connected and pooling layers which yield images from d-dimensional vectors utilizing transposed convolutional layers. On the other hand, a D acquires identical composition as G but in the opposite direction that discriminates if the input is a genuine image from a predefined dataset or whether it's an image made by G network.

DCGAN learning process is done by repeating these steps: (1) images from the dataset x are used to train the D network and (2) the generative network outputs generated insight images $G(z)$ from an arbitrary vector z as Fig. 1. Ultimately, the discriminative D network is updated from the generated image. This process aim is the point that over repeated iterations, the G network training produces images are gradually more imperceptible from the images from the dataset.

The DCGAN training is expressed as follows [22]:

$$\begin{aligned} &\underset{G}{min}\underset{D}{max}V(D,G) = \mathbb{E}_{x\sim p_{data^{(x)}}}[log D(x)] \\ &+\mathbb{E}_{z\sim p_{z^{(z)}}}[log(1-D(G(z)))], \end{aligned} \tag{1}$$

where x represents the original image, z is a d-dimensional vector made up of random numbers, and $p_{data^{(x)}}$ and $p_{z^{(Z)}}$ will be the likelihood distributions of x and z respectively. $D(x)$ represents the inputs likelihood of being an image truly produced from $p_{data^{(x)}}$. $1 - D(G(z))$ is the likelihood that it can be produced from $p_{z^{(Z)}}$. G network is trained in such

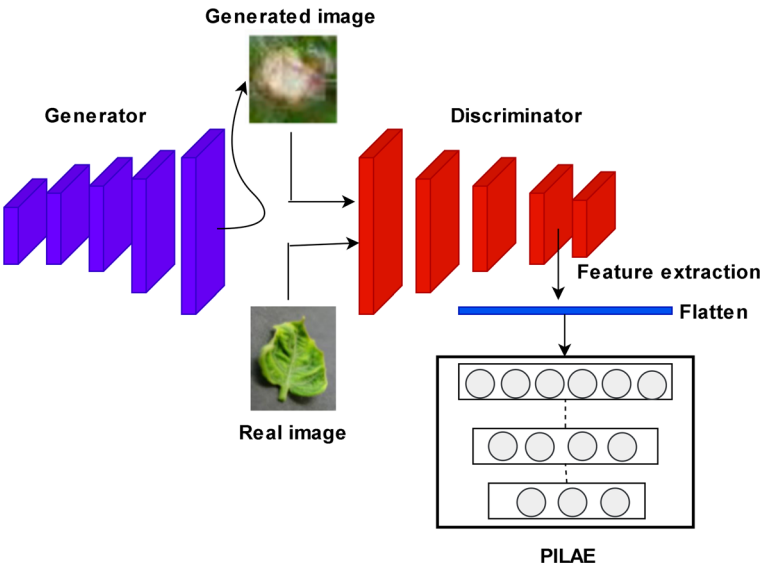

Fig. 1 DCGAN-PILAE proposed method. The DCGAN model has two function: the first is that the G network output of syntheses images are used to overcome the imbalance issue in the dataset; the second is that the D network fourth layer is used as feature extractor for both real and synthesised images. Then, these features are flattened and given to the PILAE classifier for final test accuracy

manner that it reduces $log(1 - D(G(z)))$ so that it can deceive D network. On the other hand, D network is trained so that it can increase the rate of the right answer.

After unsupervised representation is learned by using DCGAN, we then use part of the convolution layers of the discriminator as feature extraction. The extracted features are gained by transferring data items through the discriminator until certain layer f. The features matrices f are flattened and afterward trained by a classifier which uses the extracted features as input. On the other hand, the output of the G network which is a batch of generated images is used for oversampling the minor classes samples.

3.2 Pseudoinverse learning autoEncoder (PILAE)

PIL [24] is a swift supervised learning algorithm for feedforward neural network, it depends on generalized linear algebra. PIL differs from techniques that are gradient based containing BP, in that PIL is not necessarily required to adapt affined variables such as momentum, learning epoch and step length; instead, these variables are normally hard selected by the users.

PIL is utilized to train SAE with layer-wise learning framework that is greedy based; PILAE is utilized as an SAE building block. The number of hidden neurons is specified by the equation in [64]. To enable the learning of data feature, the hidden neuron's number is a slightly larger than the input network rank's and in general smaller than the input vector's dimension. The rank of the input matrix is computed by the singular value decomposition method (SVD). Truncated SVD is specifically utilized to compute the input matrix pseudoinverse, which is then utilized as the encoder weight network. On the other hand, PIL is used to compute the decoder weight matrix. Furthermore, to reduce the degree of independence parameter, the decoder and encoder weights are tied up, allowing the weight of encoder network to be equal to the decoder weight that is transposed. The autoencoder is able to map the data to high- or low-rank approximation and is used to map the data to low-rank dimensions. $X \in R^{MxN}$ is a given training set, $X = [x_1, x_2, ..., x_N]$ where the $i - th$ training sample represents as vector $\mathrm{x}^i = [x^{(1)}, x^{(2)}, ..., x^{(m)}]^T$, we utilize the SVD method to compute the pseudoinverse of the input matrix,

$$X = U \Sigma V^T \tag{2}$$

The pseudoinverse of the input matrix can calculated depend on the output of the SVD:

$$X^+ = V \Sigma' U^T, \tag{3}$$

Σ' performs the diagonal matrix which is transposed that consisted of the mutual of nonzero elements in matrix Σ. The first hidden layer neuron numbers are adjust to equal to the input matrix rank's.

The number of hidden neurons are assigned as

$$p = \beta Dim(x), \beta \in (0, 1], \tag{4}$$

Dim is function which gives the input matrix dimension, on other hand β can be an empirical parameter which is based upon the degree of the dimension to be reduced. This will assure that dimension reduction is boosted.

$\hat{X}^+$ is the low rank approximation of the matrix X^+ which calculating as follows:

$$\hat{X}^+ = \hat{V} \Sigma' U^T, \tag{5}$$

where $\hat{V}$ is shaped of the initial p rows of singular matrix V.

Since the autoencoder's input data is approximately equal to the output, we append the restriction $X = O$, where O is represents the output matrix. W_d is the decoder weight matrix and W_e is the encoder weight matrix, we assign $W_e = \hat{\mathbf{X}}^+$. Though, the optimisation objective function can rewrite as follows:

$$min\|HW - X\|^2. \tag{6}$$

Where the HW is the output of last hidden layer and the weight matrix product. The following pseudoinverse approximate solution is used to solve the optimisation problem:

$$W = H^+X. \tag{7}$$

In the case of multiple linear regression where $b = Ax + \varepsilon$, if A^TA isn't unit matrix, the least squares assessment is critical to the error in A, and x isn't significative. consequently the following equation is used to represent.

$$x = (A^TA + k)^{-1}A^Tb, k > 0. \tag{8}$$

Where k is regularization variable. When matrix column rank's is full, the orthogonal projection method is utilized to solve the pseudoinverse:

$$W^L = (H^TH)^{-1}H^TX. \tag{9}$$

The W^L is the weight matrix of layer L.

The weight decay regularization parameter is used to avert ill-posed problem:

$$W = (H^TH + kI)^{-1}H^TX. \tag{10}$$

where k is regularization restriction which the user specifies, It could be determined with the form generated in [25]. In this manner, the value of k is tune to get it the trade-off among the accuracy and generalization.

4 Proposed architecture

Currently, a lot of the algorithms use CNNs to execute both feature extraction and classification. Although in many cases impressive results are obtained, almost always there is the drawback of an exceptionally sophisticated and large network or ensemble learning, in addition to huge amounts of data. With the purpose of fully employing both PILAE and DCGAN advantages, a novel plant disease recognition framework is proposed (see Fig. 1). PILAE gets the advantage of integrating a prevailing classification efficiency, as conflicting towards the support vector machine (SVM) or CNN which BP can be used within their training, therefore DCGAN is addressed as the feature extractor and oversampling model, implying that simply after the whole DCGAN training is definitely completed just convolutional layers and the generated images are maintained. There are many reasons which makes it better select a DCGAN model: 1) The truth that it has a very swift capability to converge; 2) The generator network can efficiently fill as the intensity model of the training data, and; 3) It accomplished sampling in a very proficient and straight forward approach. The Stochastic Adam Optimizer is employed in training DCGAN since it is aimed at propping and overfitting convergence freely. Furthermore, computationally it really is proficient and is also invariant to a diagonal rescaling of the slopes, possesses little memory requirements, and is suitable for issues that are substantial as far as data and parameters are concerned.

PILAE is provided with features which are extracted by DCGAN and trained also as a feature extraction model after that employ softmax classifier. The PILAE output is given to

softmax classifier as source features. Within this paper's technique, the trained discriminator network D is utilized as a feature extractor. We utilize the same structure (see Table 1) as suggested in [38, 45]. Data enhancement isn't used, that denotes deformation approaches (flip, rotation, scale, crop, etc.) are not employed in the training set. The D network fourth layer is booked out being an extractor feature because the DCGAN is not employed to execute classification but rather used to extract deep features. An 8,192-dimensional vector space for each sample is then shaped by concatenating and flattening these features. We use the fourth layer from the D network which is the top layer before output. The intuition is that these features are linearly separable because the top layer is just a logistic regression. In general, the highest-level features are extracted in the last convolution layer [6, 47].

The feature extractor along with a classifier determines the recognition of plant disease accuracy. The higher the distinct features are, the better the classifier will be and is particularly directly proportional to the recognition rate. To begin with, BP contains the quality to be delicate to the training error surface minima. Another single layer feedforward network (SLFN) could be overtrained and therefore accomplish a generalized efficiency that is not perfect when BP learning is utilized. The PILAE output classifier includes better-generalized effectiveness when SLFN can be used to train PILAE.

4.1 Experiments and analysis

Three datasets are used to validate the proposed model: 1) the PlantVillage dataset [28], 2) Swedish leaf dataset [53] and 3) Leafsnap dataset [34].

4.1.1 Evaluation on PlantVillage dataset

The PlantVillage dataset [28] was used to evaluate the proposed method. The dataset consists of 54,323 samples of 14 crop species with 38 classes of healthy or diseases plants. In

Table 1 The structure of generator G and discriminator D of DCGAN [45], B refers to the batch size. The input of G network is a noise from a normal distribution; for the D network, it is the sample's shape from the real dataset

Generator	
input shape = (B, 100)	
Layer type	output shape
transposed conv	(B, 512, 4, 4)
transposed conv	(B, 256, 8, 8)
transposed conv	(B, 128, 16, 16)
transposed conv	(B, 64, 32, 32)
transposed conv	(B, 3, 64, 64)
Discriminator	
input shape = (B, 3,64,64)	
conv	(B, 64, 32, 32)
conv	(B, 128, 16, 16)
conv	(B, 256, 8, 8)
conv	(B, 512, 4, 4)
dense layer	(B, 1)

Fig. 2, we can observe that some classes from plant images lead to an imbalance problem, such as the categories of potato healthy, peach healthy, raspberry healthy and tomato yellow leaf curl virus compared to tomato healthy (for example, the ratio of the potato healthy class to the tomato healthy class is 1:33 because the potato healthy class has 152 samples and the tomato healthy has 5,375 samples). In such cases, the test images are at risk of being mismatched. Thus, we utilized the output images from generator network G to oversample these minor class. The 14 classes that have samples $<$1,000 samples are oversampled so that they have 1,000 samples; thus, the final ratio of minor: major will be approximately equal to 1:5, with a final size of 60,344. Two experiments are conducted: the first with the original dataset and the second with the balanced dataset. The hyperparameters of DCGAN are set to the same work of [45], after preliminary experiments, we found it is the best to set the learning rate to 3e-3 with a batch size of 128 and we train for 15 epochs where each epoch take 12 minutes. The PILAE algorithm is given the extracted features from DCGAN and trained also as features extraction and classifier to give the final test accuracy. The dataset is split randomly into training and testing with different ratios (80%:20%, 70%:30% and 60%:40% where the first number refers to training and the second to testing ratio). All experiments are done on a computer with the Intel core i7-6800k CPU, one GPU GTX1080. We performed our framework in MATLAB, utilizing the MatConvNet library [55, 61] for our execution of DCGAN-PILAE model. The suggested method is weighed against released methods whose computation settings is similar or more than ours.

In Table 2, we can see the result of the proposed model in the first experiment (original dataset), in which different split ratios were used to reduce variability. As the ratio of testing size increases, more time is consumed for classification. (Time is required for extracting features by DCGAN and the PILAE classifier.) The best results were obtained using a 70:30 ratio; however, compared to other methods that used the same dataset, our results were inferior due to the imbalanced problem in the dataset. In the second experiment, we utilized the full function of DCGAN in which the discriminator network D is used to extract the features of both real and generated images (images were generated for the minor classes only). Next, the PILAE was trained over these features to determine the test accuracy. The

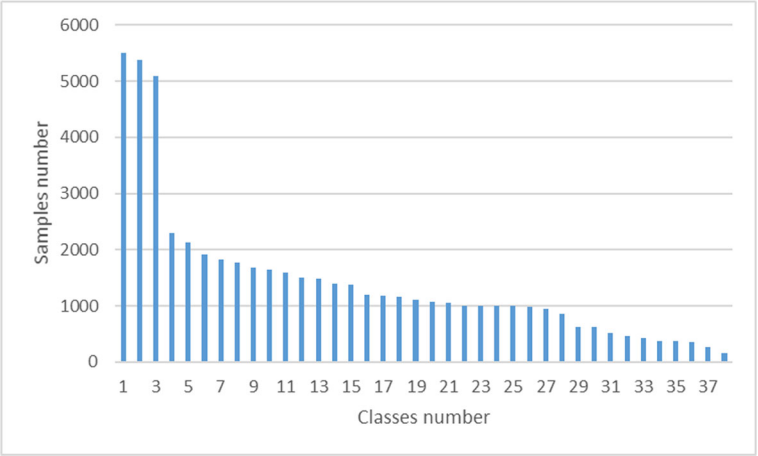

Fig. 2 Number of samples per class in plant disease dataset

Table 2 Results for proposed method with different split ratios. * indicates the balanced dataset, and time in hours

Train-Test	Training accuracy	Testing accuracy	Time
80-20	97.34	94.95	2.5
80-20*	99.98	100	2.6
70-30	98.89	97.97	2.8
70-30*	99.66	100	2.9
60-40	98.64	97.29	3.1
60-40*	99.95	99.29	3.2

best results were obtained using an 80:20 ratio; therefore, we utilized this ratio for comparing with other state-of-the-art methods that used the same dataset. The training set includes 48,275 samples, while the test set includes 12,069 samples.

The best results for the dataset were achieved using 7 different algorithms [3, 10, 19, 36, 40, 51, 58, 59]:

1. In [10], the authors employed different DL models, such as Inception_v3 and ResNet34. The authors added one additional class to the dataset, a 715-sample background class, so that the classifier could be trained to differentiate between plant leaves and the surrounding environment. This class was created using color samples from Stanford open public dataset of background samples. Thus, the size of the dataset after including the background class became 55,038, splits into 39 classes.
2. In [58], the authors used the original dataset and employed different DL methods, including Inception V4 and DenseNet.
3. In [40], the authors applied different train: test ratios and employed two DL methods: AlexNet and GoogLeNet.
4. Liang et al. introduced a model that included three DL models, ResNet50, WideResNet50, and Dual Path Network (DPN92), for classification of plant diseases [36].
5. Shi et al. [51] proposed an approach for plant diseases classification that merged a rectification module and rectified meta-learning to learn from noisy labels. This framework comprised of two steps, the primary learning step and the rectified meta-learning step. The rectified meta-learning step was adapted to make the primary learning step less susceptible to overfitting to label noise.
6. In [59], a pruned model based on DenseNet was suggested. The authoress found that pruning model helped in removing unimportant and redundant connections; hence, its focus is efficiency. To further improve the efficiency of the model, they applied self-normalization neural network (SNN) model with scaled exponential linear unit (SELU), Alphadropout, and initialization scheme.
7. Ale et al. [3] presented a DenseNet-based transfer learning model for plant disease detection. To further decrease the size of the model, they suggested a lightweight DL model that can run on edge devices with limited executing capacities, which are based on and extend the DNN model structure by decreasing the number of neurons and layers in each layer or the size of input samples.

To rank the execution of the proposed strategy, we ascertain different execution scores; we have utilized three distinct scores to measure the model execution. They are Accuracy,

Recall, and Precision. Accuracy is the definitive prominent score in any classification problem in which the classes have roughly the same ratio of samples (balanced). It is defined by

$$(TN + TP)/(FP + FN + TP + TN) \tag{11}$$

where FP, TN, TP, and FN are the number of false positive, true negative, true positive, and false negative samples, respectively; the maximum accuracy value is 1, indicating that there no misclassification of samples in the dataset, and 0 is the lowest possible value. We performed the experiments five times and calculated the average accuracy for each method. Recall is given as TP/(TP+FN), depicts the positive samples ratio recovered in the test set, A perfect classifier will have a recall estimation of 1 and the lower rate for an outrageous classifier would be 0. Precision is identified as TP/(TP+FP). Precision is the ratio of TRUE class perceptions classified effectively to the absolute number of perceptions that were classified as positive. Similar to recall, even here the worst rate is 0 and the best rate is 1.

In Table 3, It is shown that the proposed method is ahead of the other seven methods in terms of test accuracy. For the first method in the above list [10], the configuration is higher than ours by four GB of RAM size of GPU, but our training time still lower than theirs even though the proposed method has two terms (DCGAN and PILAE algorithm). Our proposed model exceeds the second method [58] in both test accuracy and training time, even though the GPU utilized in this method is two GB larger than ours in RAM size. For the third method, the authors mentioned that the method consumed many hours on a high-execution GPU cluster computer in which the classification time is very rapid (less than a second on a CPU); thus, this method exceeds our model in terms of classifier time because the PILAE algorithm in our configuration takes 0.25 hours to execute final results. Although the sixth method [59] has lower test accuracy than the presented work, it consumes less training time (nearly the half that of our method). We attribute this to the light based on pruning that was used in this method, as well as the configuration, which is higher than ours in the size of GPU RAM. Thus, the DCGAN-PILAE achieves the balance between computational efficiency and test accuracy. To further support the efficiency of the proposed model, we provide the accuracy for every class in the dataset by listing the recall and precision scores in Table 4.

Table 3 Result comparisons for PlantVillage dataset with best methods

(Best) CNN architecture	Test accuracy	Training time [h]	Configuration	Accuracy of separate classes
Inception_v3 [10]	99.76	5.64	CPU: I7-4790 GPU: 1×GTX TITAN X	–
DenseNet [58]	99.75	18.04	GPU: 1×Tesla K40c	–
GoogLeNet [40]	99.34	–	–	–
Integrated [36]	99.92	–	GPU: 1×GTX 1080 Ti	–
Rectified meta-learning[51]	99.52	–	–	–
DenseNet-161 (pruned)[59]	99.81	1.19	1×Tesla K40c	–
DenseNet-121 [3]	98.00	–	CPU: i7−8700 GPU: 1×GTX 1070 Ti	–
DCGAN-PILAE	100	2.6 (0.25 for PILAE)	CPU: i7−6800k GPU: 1×GTX 1080	99.99−100

Table 4 Performance of each class on PlantVillage dataset

#	Class	Precision	Recall
1	Apple black rot	1	1
2	Apple cedar rust	1	1
3	Apple healthy	1	1
4	Apple scab	1	1
5	Blueberry healthy	1	1
6	Cherry healthy	1	1
7	Cherry powdery mildew	1	1
8	Corn grey leaf spot	1	1
9	Corn common rust	1	1
10	Corn healthy	1	1
11	Corn northern leaf blight	1	1
12	Grape black rot	1	1
13	Grape black measles (Esca)	1	1
14	Grape healthy	0.99	1
15	Grape leaf blight	1	1
16	Orange Huanglongbing (Citrus Greening)	1	1
17	Peach bacterial spot	1	1
18	Peach healthy	1	1
19	Bell pepper bacterial spot	1	1
20	Bell pepper healthy	1	1
21	Potato early blight	1	0.99
22	Potato healthy	1	1
23	Potato late blight	1	1
24	Raspberry healthy	1	1
25	Soybean healthy	1	1
26	Squash powdery mildew	1	1
27	Strawberry healthy	1	1
28	Strawberry leaf scorch	1	1
29	Tomato bacterial spot	1	1
30	Tomato early blight	1	1
31	Tomato late blight	1	1
32	Tomato leaf mould	1	1
33	Tomato septoria leaf spot	1	1
34	Tomato two spotted spider mite	1	1
35	Tomato target spot	1	1
36	Tomato mosaic virus	1	1
37	Tomato yellow leaf curl virus	1	1
38	Tomato healthy	0.99	1

For PILAE algorithm, in Fig. 3a the impact of β values regarding the accuracy of prediction on the dataset is given by $\beta = .4$. By examining the accuracy curve, it is clear that there are small changes as the parameters of β vary. This result implies that the empirical parameter β necessitates little influence on prediction performance. We compute the ratio between rank and the input data dimension in each layer in Fig. 3b, where the test error curve on dataset being illustrated in Fig. 3c. The test error is a classification error over the test set. From Fig. 3c it can be identified that the test error reduces while the network goes deeper. Finally, in Fig. 3d we show the effect of regularization parameter k regarding testing accuracy. The k values vary from $[10^{-6}, 10^{-5}, ..., 10^{5}, 10^{6}]$. It can be observed that $k = 10$ yields the best accuracy (100%).

4.1.2 Evaluation on Swedish leaf dataset

The Swedish leaf is balanced dataset [53] that includes 1,125 leaf samples from 15 diverse Swedish trees, with 75 samples per class. We made a few changes to ensure that it was imbalanced: we chose classes 1 to 7 as the minority classes and randomly chose 15 samples from each of these classes (105 samples). The other classes (8 to 15) were designated as the majority classes. This gave the dataset a minor: major ratio equal of 1:6. Thus, we applied the G network of DCGAN to generate samples for minor classes to overcome the imbalanced issue, which results in final dataset of 1,125 samples, and applied D network to extract the features for both real and generated samples. The PILAE algorithm was applied over these samples as a classifier.

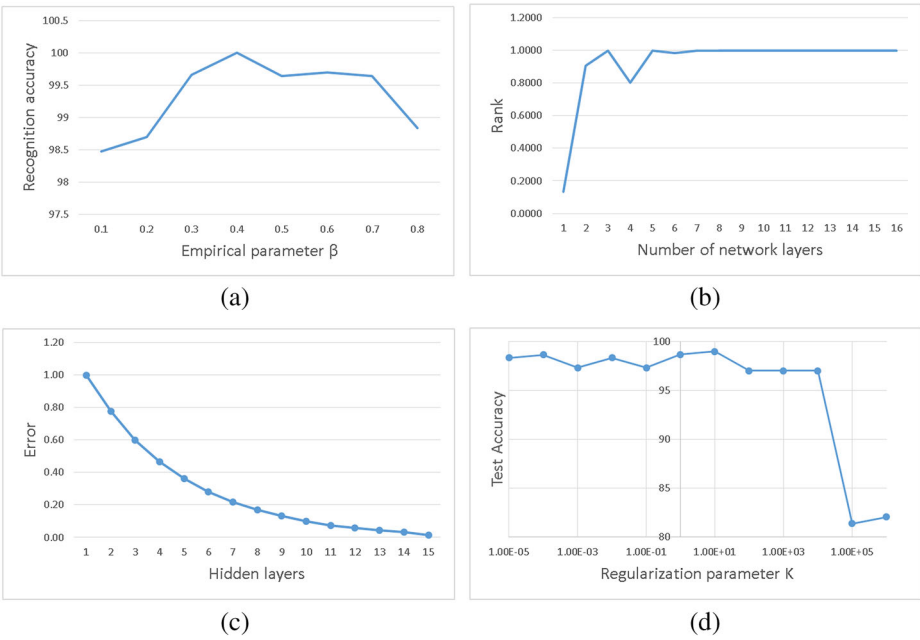

Fig. 3 Visualization of PILAE algorithm curves on PlantVillage dataset: **a** Test accuracy curve regarding to β, **b** The rank ratio as the model layers increased, **c** The error as the model layers increased, and **d** The effect of regularization parameter k to the test accuracy

Table 5 Results of different split ratios for Swedish leaf dataset. Time in seconds

Train-Test	Training accuracy	Testing accuracy	Time
80-20	99.02	99.00	173.5
70-30	98.66	98.43	194.65
60-40	99.82	99.32	214.56

For the hyperparameters of DCGAN and PILAE, we applied the same experiment as performed on the first dataset, setting the learning rate to 4e-3, and trained for the same number of epochs (each epoch takes 22 seconds). We ran the experiment on different data split ratios to reduce variability (Table 5). The best results were obtained using a 60:40 ratio; therefore, we utilized this ratio to compare our method with other state-of-the-art methods that used the same dataset. The training set includes 675 samples, while the test set includes 450 samples.

The best results for the Swedish leaf dataset were achieved using four different methods; Elliptical-Half-Gabor (EHG) [62], Siamese network [68], Triangle-Distance-Representation (TDR) [70], and pruned network [59]. In the EHG, a new type of Gabor wavelet, the elliptical half Gabor wavelet, is used to focus on the local dominant orientation data of veins, leaf contours, and main textures. Next, they apply an enhanced local line direction coding technique called the maximum gap local line direction pattern (MGLLDP) to extract local dominant orientation system patterns from the elliptical half Gabor wavelet domain. The histogram of the normalized local dominant orientation system patterns is the counting-based local system descriptor, and SVM is used as the classifier. In the second method [68], the Siamese network is presented to form a metric space for leaf classification in which dissimilar samples are far away from each other and similar samples are close to each other. Furthermore, a spatial structure optimizer (SSO) approach for forming the metric space is applied, which improves the accuracy of leaf classification. Finally, a k-nearest neighbor (kNN) classifier is employed to classify leaves in the learned metric space.

The third method [70] is a new shape description technique named triangle-distance representation (TDR) for plant leaf recognition. The TDR descriptor is defined by two matrices: a triangle center distance matrix and a sign matrix. The triangle center distance matrix is utilized to illustrate the bending degree and spatial information of a shape contour, while the sign matrix is utilized to describe the convex/concave property of the shape contour. The last method is the pruned network [59] which is described above, as we used it for comparison for the first dataset.

In Table 6, the evaluation of the test accuracy between the reported results and the proposed method for the Swedish leaf dataset are illustrated. This reveals that our DCGAN-PILAE

Table 6 Result comparisons for Swedish leaf dataset

(Best) CNN architecture	Test accuracy	Training time [S]	Configuration
EHG [62]	98.40	–	–
S-Inception [68]	91.67	–	CPU: i7-6700HQ GPU: 1×GTX 1060
(TDR) [70]	97.27	–	CPU: i7-8700K
DenseNet-121 [59]	100	2250	GPU: 1×Tesla K40c
DCGAN-PILAE	99.32	214.56 (16.78 for PILAE)	CPU: i7-6800k GPU: 1×GTX 1080

model is the second-most accurate out of the five total methods, taking into account the configuration of the compared methods. Compared with the pruned network model [59], which achieved test accuracy of 100% , the DCGAN-PILAE has a training time that is 8 times shorter. Thus, DCGAN-PILAE model is balanced between test accuracy and time.

4.1.3 Evaluation on leafsnap dataset

Leafsnap is a dataset that consists of a collection of leaves samples of 185 species, which are visualized in two techniques. The first technique is the samples that have been scanned in laboratory conditions. In the second technique, plant species are visualized in a bright background with a mobile device and under different conditions, such as different light, shading, noise, and blur. This dataset has been employed to design mobile application for recognizing leaf samples [34]. This dataset contains 23,145 lab samples and 7,719 field samples, in which we utilized the field images for training and evaluation the proposed method.

For the hyperparameters of DCGAN and PILAE, we applied the same experiment as performed on the second dataset, setting the learning rate to 4e-3, and we trained for the same number of epochs (15 epochs; each epoch takes 92 seconds). The PILAE regularization parameter K is set to one. We ran the experiment on different data split ratios like the first two datasets. The best results were obtained using a 80:20 ratio; therefore, we utilized this ratio to compare our method with other state-of-the-art methods that used the same dataset. The training set includes 6,175 samples, while the test set includes 1,544 samples.

We evaluate seven leaf classification models; generalised procrustes analysis (GPA) [14], GIST texture features [31], Dual-Path CNN [50], LeafNet [7], Siamese network [68], pruned network [59] and our proposed model DCGAN-PILAE.

For the first method [14], a leaf recognition model that depended on contour unwrapping and leaf apex alignment was proposed, which effectively defeats the traditional GPA weakness to align contours with variations in translation, rotation and scale. In the second method [31], the Cosine KNN classifier and PCA algorithm in the gist feature vector of leaf samples was applied. The third method [50] employed a dual-path deep CNN to learn leaf image features and optimize for classification of the leaf based on shape-texture dual-path deep CNN and marginalized shape context. For the fourth method [7], an architecture similar to well-known CNN models like AlexNet was proposed for the classification of leaf samples. The last two method [59, 68] are described above, as we used them to compare the first and second datasets.

In Table 7, the test accuracy evaluation between reported results and the proposed method is illustrated for the Leafsnap dataset. It is seen that the proposed method achieves the first position between all the other six methods [7, 14, 31, 50, 59, 68] in conditions of test accuracy. DCGAN-PILAE exceeds the pruned network [59] in training time, although its configuration is higher than our proposed model. For LeafNet [7], it's configuration is less than our moel, so it may have a training time below our reported one. The remaining reported results do not have the configuration or the training time or both of them. Finally, DCGAN-PILAE is balanced between test accuracy and time complexity.

4.2 Discussion

The experiments described above have proven that the proposed combination of DCGAN and PILAE attains an excellent accuracy performance compared to other models. DCGAN was utilized in our case because it can extract the relevant features in an unsupervised way,

Table 7 Result comparisons for Leafsnap dataset

(Best) CNN architecture	Test accuracy	Training time [H]	Configuration
GPA [14]	84.80	9.6	CPU: i7
GIST texture features [31]	74.50	2.01	–
Dual-Path CNN [50]	95.61	–	Nvidia GPU
LeafNet [7]	86.30	0.9	GPU: NVIDIA GTX 960
S-Inception [68]	95,75	–	CPU: i7−6700HQ
			GPU: 1×GTX 1060
DenseNet-161 [59]	92.33	10.34	GPU: 1×Tesla K40c
DCGAN-PILAE	97.86	0.4	CPU: i7−6800k
		(0.03 for PILAE)	GPU: 1×GTX 1080

without requiring expert analysis of the learning process, and the generated images from the G network are used to oversample the minor classes.

Comparing the results of our proposed model for the balanced and imbalanced datasets (Table 2), we found that our classifier PILAE gives the best test accuracy for the balanced dataset, which means that PILAE suffers from the imbalanced issue. Nonetheless, PILAE has a good training time in the imbalanced dataset (Table 2), with suitable test accuracy.

The computation complexity of our model has two terms. First, for DCGAN, it is $O(B^2)$ where B is the batch size of discriminator and generator networks. Second, the PILAE algorithm can be approximately clarified as $\mathcal{O}(dN^2)$ [26] where N indicates the number of samples in dataset and d is the dimension of the sample. Although our method has two parts in terms of computational complexity, it achieves good results comparing to other models. For the ratio of 80%:20% one epoch of DCGAN model takes 12 minutes, and the PILAE algorithm takes 15 minutes to classify the balanced dataset.

PILAE classifier possess a fast training time because of: 1) it does not require fine-tuning; 2) PILAE weights can be analytically identified, unlike in traditional autoencoders where iterative algorithms are essentially required; and 3) it learns to signify features through singular values unlike autoencoders where the representation of data is learned. Also, the high classification ratios in suitable processing times, enforced by the fact that our model has been executed effectively using GPUs, the utilization of GPUs allow us to realize the full probable of DCGAN approach for feature extraction and synthesis images followed by PILAE structure. The proposed method thus gets the balance better between computational performance and recognition ability as compared to other classical methods.

From Table 3 and 6, we can see that the proposed method achieved a test accuracy 100% on the PlantVillage dataset, where the pruned network model [59] reaches the test accuracy of 100% on the Swedish leaf dataset. Thus, based on the optimum test accuracy (100%) for the two datasets was already achieved, our future work will focus on extending our model to work on edge devices (mobile phone, drones) through insertion some advanced detection models [18, 20, 71, 74].

5 Conclusion

In presented work, the functions of DCGAN model and the PILAE algorithm have been designed for extracting features and oversampling the minor samples (DCGAN) and

classifying ailments that impact plant leaves (PILAE). The empirical results demonstrate that the presented model is a valuable model that may support a precise recognition of leaf ailments, using little computational energy and achieved better accuracy than many previously proposed models. An expansion of this work will concentrate on PILAE development to resolve the dataset imbalance problem without requiring any additional methods.

References

1. Akhtar A, Khanum A, Khan SA, Shaukat A (2013) "Automated plant disease analysis (apda): Performance comparison of machine learning techniques". In: 2013 11th International Conference on Frontiers of Information Technology, pp. 60–65 IEEE
2. Al-Hiary H, Bani-Ahmad S, Reyalat M, Braik M, ALRahamneh Z (2011) Fast and accurate detection and classification of plant diseases. International Journal of Computer Applications 17(1):31–38
3. Ale L, Sheta A, Li L, Wang Y, Zhang N (2019) "Deep learning based plant disease detection for smart agriculture". In: 2019 IEEE Globecom Workshops GC Wkshps, pp. 1–6 IEEE
4. Arsenovic M, Karanovic M, Sladojevic S, Anderla A, Stefanovic D (2019) Solving current limitations of deep learning based approaches for plant disease detection. Symmetry 11(7):939
5. Athanikar G, Badar P (2016) Potato leaf diseases detection and classification system. International Journal of Computer Science and Mobile Computing 5(2):76–88
6. Athiwaratkun B, Kang K (2015) "Feature representation in convolutional neural networks". arXiv:1507.02313
7. Barré P., Stöver B. C., Müller K. F., Steinhage V (2017) Leafnet: a computer vision system for automatic plant species identification. Ecological Informatics 40:50–56
8. Barua S, Ma X, Erfani SM, Houle ME, Bailey J (2019) Quality evaluation of gans using cross local intrinsic dimensionality. arXiv:1905.00643
9. Blancard D, Laterrot H, Marchoux G, Candresse T (2012) 2-Diagnosis of parasitic and nonparasitic diseases. Tomato Diseases 35–411
10. Brahimi M, Arsenovic M, Laraba S, Sladojevic S, Boukhalfa K, Moussaoui A (2018) Deep Learning for Plant diseases: Detection and Saliency Map Visualisation, pp. 93–117 Springer International Publishing
11. Brahimi M, Boukhalfa K, Moussaoui A (2017) Deep learning for tomato diseases: classification and symptoms visualization. Appl Artif Intell 31(4):299–315
12. Cai W, Wei Z (2020) Piigan: Generative adversarial networks for pluralistic image inpainting. IEEE Access 8:48451–48463
13. Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) Smote: synthetic minority over-sampling technique. Journal of artificial intelligence research 16:321–357
14. Choudhury SD, Yu J-G, Samal A (2018) Leaf recognition using contour unwrapping and apex alignment with tuned random subspace method. Biosyst Eng 170:72–84
15. Dandawate Y, Kokare R (2015) "An automated approach for classification of plant diseases towards development of futuristic decision support system in indian perspective. In: 2015 International conference on advances in computing, communications and informatics (ICACCI), pp. 794–799 IEEE
16. DeChant C, Wiesner-Hanks T, Chen S, Stewart EL, Yosinski J, Gore MA, Nelson RJ, Lipson H (2017) Automated identification of northern leaf blight-infected maize plants from field imagery using deep learning. Phytopathology 107(11):1426–1432
17. Dyrmann M, Karstoft H, Midtiby HS (2016) Plant species classification using deep convolutional neural network. Biosyst Eng 151:72–80
18. Fan DP, Cheng MM, Liu JJ, Gao SH, Hou Q, Borji A (2018) "Salient objects in clutter:, Bringing salient object detection to the foreground". In: Proceedings of the European conference on computer vision (ECCV), pp 186–202
19. Ferentinos KP (2018) Deep learning models for plant disease detection and diagnosis. Comput Electron Agric 145:311–318
20. Fu K, Zhao Q, Gu IY-H, Yang J (2019) Deepside: a general deep framework for salient object detection. Neurocomputing 356:69–82
21. Fujita E, Kawasaki Y, Uga H, Kagiwada S, Iyatomi H (2016) Basic investigation on a robust and practical plant diagnostic system. In: 2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 989–992 IEEE
22. Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial nets. In: Advances in neural information processing systems, pp 2672–2680

23. Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial nets. In: Advances in neural information processing systems, pp 2672–2680
24. Guo P, Lyu MR (2004) A pseudoinverse learning algorithm for feedforward neural networks with stacked generalization applications to software reliability growth data. Neurocomputing 56:101–121
25. Guo P, Lyu MR, Chen CLP (2003) Regularization parameter estimation for feedforward neural networks. IEEE Transactions on Systems, Man, and Cybernetics, Part B 33(1):35–44
26. Guo P, Zhou X, Wang K (2018) "Pilae:, A non-gradient descent learning scheme for deep feedforward neural networks". arXiv:1811.01545
27. Hanssen IM, Lapidot M (2012) "Major tomato viruses in the mediterranean basin," In: Advances in virus research, vol. 84, pp. 31–66 Elsevier
28. Hughes D, Salathé M. et al (2015) "An open access repository of images on plant health to enable the development of mobile disease diagnostics". arXiv:1511.08060
29. Hwang U, Choi S, Yoon S (2017) Disease prediction from electronic health records using generative adversarial networks. arXiv:1711.04126
30. Kawasaki Y, Uga H, Kagiwada S, Iyatomi H (2015) "Basic study of automated diagnosis of viral plant diseases using convolutional neural networks". In: International Symposium on Visual Computing pp. 638–645 Springer
31. Kheirkhah FM, Asghari H (2018) Plant leaf classification using gist texture features. IET Comput Vis 13(4):369–375
32. Kornblith S, Shlens J, Le QV (2019) Do better imagenet models transfer better? In: proceedings of the IEEE Conference on Computer Vision and Pattern Recognition 2661–2671
33. Krizhevsky A, Sutskever I, Hinton GE (2012) "Imagenet classification with deep convolutional neural networks". In: Advances in neural information processing systems, pp 1097–1105
34. Kumar N, Belhumeur PN, Biswas A, Jacobs DW, Kress WJ, Lopez I, Soares JVB (2012) "Leafsnap: A computer vision system for automatic plant species identification". In: The 12th European Conference on Computer Vision ECCV
35. LeCun Y, Bengio Y, Hinton G (2015) "Deep learning". nature 521(7553):436
36. Liang S, Zhang W (2019) "Accurate image recognition of plant diseases based on multiple classifiers integration". In: Chinese Intelligent Systems Conference, pp. 103–113 Springer
37. Lu Y, Yi S, Zeng N, Liu Y, Zhang Y (2017) Identification of rice diseases using deep convolutional neural networks. Neurocomputing 267:378–384
38. Mahmoud MAB, Guo P (2019) A novel method for traffic sign recognition based on dcgan and mlp with pilae algorithm. IEEE Access 7:74602–74611
39. Mao W, Liu Y, Ding L, Li Y (2019) Imbalanced fault diagnosis of rolling bearing based on generative adversarial network: a comparative study. IEEE Access 7:9515–9530
40. Mohanty SP, Hughes DP, Salathé M. (2016) Using deep learning for image-based plant disease detection. Frontiers in plant science 7:1419
41. Mokhtar U, El Bendary N, Hassenian AE, Emary E, Mahmoud MA, Hefny H, Tolba MF (2015) "Svm-based detection of tomato leaves diseases". In: Intelligent Systems' 2014, pp. 641–652 Springer
42. Nachtigall LG, Araujo RM, Nachtigall GR (2016) "Classification of apple tree disorders using convolutional neural networks". In: 2016 IEEE 28th International Conference on Tools with Artificial Intelligence ICTAI, pp. 472–476 IEEE
43. Nazki H, Yoon S, Fuentes A, Park DS (2020) "Unsupervised image translation using adversarial networks for improved plant disease recognition". Computers and Electronics in Agriculture 168:105117
44. Ping G, Duan F, Pei W, Yao Y, Xin X (2017) Pulsar candidate identification with artificial intelligence techniques. arXiv:1711.10339
45. Radford A, Metz L, Chintala S (2015) "Unsupervised representation learning with deep convolutional generative adversarial networks". arXiv:1511.06434
46. Ren S, He K, Girshick R, Sun J (2015). In: Advances in neural information processing systems, pp 91–99
47. Ren JS, Wang W, Wang J, Liao S (2012) "An unsupervised feature learning approach to improve automatic incident detection". In: 2012 15th International IEEE Conference on Intelligent Transportation Systems pp. 172–177 IEEE
48. Sa I, Ge Z, Dayoub F, Upcroft B, Perez T, McCool C (2016) Deepfruits: a fruit detection system using deep neural networks. Sensors 16(8):1222
49. Samanta D, Chaudhury PP, Ghosh A (2012) Scab diseases detection of potato using image processing. International Journal of Computer Trends and Technology 1:3
50. Shah MP, Singha S, Awate SP (2017) "Leaf classification using marginalized shape context and shape+texture dual-path deep convolutional neural network". In: 2017 IEEE International Conference on Image Processing ICIP, pp. 860–864 IEEE
51. Shi R, Zhai D, Liu X, J Jiang et al (2020) "Rectified meta-learning from noisy labels for robust image-based plant disease diagnosis". arXiv:2003.07603

52. Sladojevic S, Arsenovic M, Anderla A, Culibrk D, Stefanovic D (2016) Deep neural networks based recognition of plant diseases by leaf image classification. Computational intelligence and neuroscience. 2016
53. Söderkvist O. (2001) "Computer vision classification of leaves from swedish trees" Master's Thesis, Teknik Och Teknologier Stockholm Sweden
54. Suh S, Lee H, Jo J, Lukowicz P, Lee YO (2019) Generative oversampling method for imbalanced data on bearing fault detection and diagnosis. Applied Sciences, vol. 4:9
55. SungHo B "Deep convolutional generative adversarial network (dcgan) implementation on matconvnet." https://github.com/sunghbae/dcgan-matconvnet, [Online; accessed 19-6-2019].
56. Szegedy C, Vanhoucke V, Ioffe S, Shlens J, Wojna Z (2016) Rethinking the inception architecture for computer vision. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 2818–2826
57. Tian Y, Yang G, Wang Z, Li E, Liang Z (2019) Detection of apple lesions in orchards based on deep learning methods of cyclegan and yolov3-dense, vol 2019
58. Too EC, Li Y, Njuki S, Liu Y (2018) A comparative study of fine-tuning deep learning models for plant disease identification. Comput Electron Agric 161:272–279
59. Too EC, Yujian L, Kwao P, Njuki S, Mosomi ME, Kibet J (2019) Deep pruned nets for efficient image-based plants disease classification. Journal of Intelligent & Fuzzy systems, no Preprint 1–17
60. Valerio Giuffrida M, Scharr H, Tsaftaris SA (2017) "Arigan: Synthetic arabidopsis plants using generative adversarial network". In: Proceedings of the IEEE International Conference on Computer Vision Workshops pp 2064–2071
61. Vedaldi A, Lenc K (2015) "Matconvnet: Convolutional neural networks for matlab". In: proceedings of the 23rd ACM international conference on Multimedia, pp. 689–692 ACM
62. Wang X, Du W, Guo F, Hu S (2020) Leaf recognition based on elliptical half gabor and maximum gap local line direction pattern. IEEE Access 8:39175–39183
63. Wang J, Guo P, Xin X (2018) "Review of pseudoinverse learning algorithm for multilayer neural networks and applications". In: International Symposium on Neural Networks, pp. 99–106 Springer
64. Wang K, Guo P, Xin X, Ye Z (2017) "Autoencoder low rank approximation and pseudoinverse learning algorithm". In: Systems, Man, and Cybernetics SMC, 2017 IEEE International Conference on, pp. 948–953 IEEE
65. Wang BX, Japkowicz N (2004) "Imbalanced data set learning with synthetic samples". In: Proc. IRIS Machine Learning Workshop 19 sn
66. Wang H, Li G, Ma Z, Li X (2012) "Application of neural networks to image recognition of plant diseases". In: 2012 International Conference on Systems and Informatics ICSAI 2012, pp. 2159–2164 IEEE
67. Wang D, Vinson R, Holmes M, Seibel G, Bechar A, Nof S, Luo Y, Tao Y (2018) "Early tomato spotted wilt virus detection using hyperspectral imaging technique and outlier removal auxiliary classifier generative adversarial nets (or-ac-gan)". In: 2018 ASABE Annual International Meeting, p. 1 American Society of Agricultural and Biological Engineers
68. Wang B, Wang D (2019) "Plant leaves classification: A few-shot learning method based on siamese network". IEEE Access 7:151754–151763
69. Wang Z, Zou C, Cai W (2020) Small sample classification of hyperspectral remote sensing images based on sequential joint deeping learning model. IEEE Access 8:71353–71363
70. Yang C, Wei H (2019) Plant species recognition using triangle-distance representation. IEEE Access 7:178108–178120
71. You H, Tian S, Yu L, Lv Y (2019) "Pixel-level remote sensing image recognition based on bidirectional word vectors". In: IEEE Transactions on Geoscience and Remote Sensing
72. Zhang J, Chen L, Zhuo L, Liang X, Li J (2018) An efficient hyperspectral image retrieval method: Deep spectral-spatial feature extraction with dcgan and dimensionality reduction using t-sne-based nm hashing. Remote Sens 10(2):271
73. Zhang Z, Liu X, Cui Y (2016) Multi-phase offline signature verification system using deep convolutional generative adversarial networks. In: 2016 9th international Symposium on Computational Intelligence and Design (ISCID), vol. 2, pp. 103–107 IEEE
74. Zhao J-X, Liu J-J, Fan D-P, Cao Y, Yang J, Cheng M-M (2019) "Egnet:, Edge guidance network for salient object detection". In: Proceedings of the IEEE International Conference on Computer Vision, pp 8779–8788
75. Zhu Y, Aou M, Krijn M, Vanschoren J, Campus HT (2018) "Data augmentation using conditional generative adversarial networks for leaf counting in arabidopsis plants" BMVC p 324

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.